application code. To get status upgrades to work, and the tests to pass, you need to write some application code. For example, you could make the following changes to the `FrequentFlyer` class:

```
StatusService statusService;

public void setStatusPoints(int statusPoints) {
    this.statusPoints = statusPoints;
    updateStatusLevel();
}

private void updateStatusLevel() {
    setStatus(statusService.statusLevelFor(statusPoints));
}
```

You need a service to tell you what status can be obtained for a given number of points.

Update the status level to the appropriate level.

Here you've discovered the need for a new service (`StatusService`) and method (`statusLevelFor()`). This service needs to provide the status level that corresponds to a given number of status points.

Notice how the principle of outside-in development isn't limited to tests. When you write implementation code, you can use the same principle, writing the code you'd like to have, and use this process to discover the classes and methods that you need. Whenever you write a piece of code that doesn't yet exist, you're discovering a new low-level requirement.

In general, when you come across something you need from another class or method, you have several choices:

- Implement the class or method immediately.
- Implement a minimum version of the class, and come back to it later.
- Defer implementation by using a "fake" class (a stub or mock) until your current test works, and then go back to implement the class.

Each of these approaches has advantages and trade-offs; experienced practitioners typically know how to use each, and how to pick the most appropriate approach for a given situation. In the following sections we'll look at each of these approaches briefly.

10.3.4 Implement simple classes or methods immediately

Specifying and implementing immediately works well for simple, obvious implementations that can be done quickly. This is what you did in the previous section, where you implemented the `Status` class and the `getStatus()` method of the `FrequentFlyer` class directly, without needing any dedicated unit tests.

Oftentimes when you start to implement an apparently simple method or class, you realize that it's more complicated than you initially thought. If a class has any nontrivial behavior, it's a good idea to describe this behavior through "executable specification"-style unit tests.

To do this, you need to put your current test on standby while you implement the new method or class. For example, in JUnit, you might use the `@Ignore` annotation to temporarily skip this test: